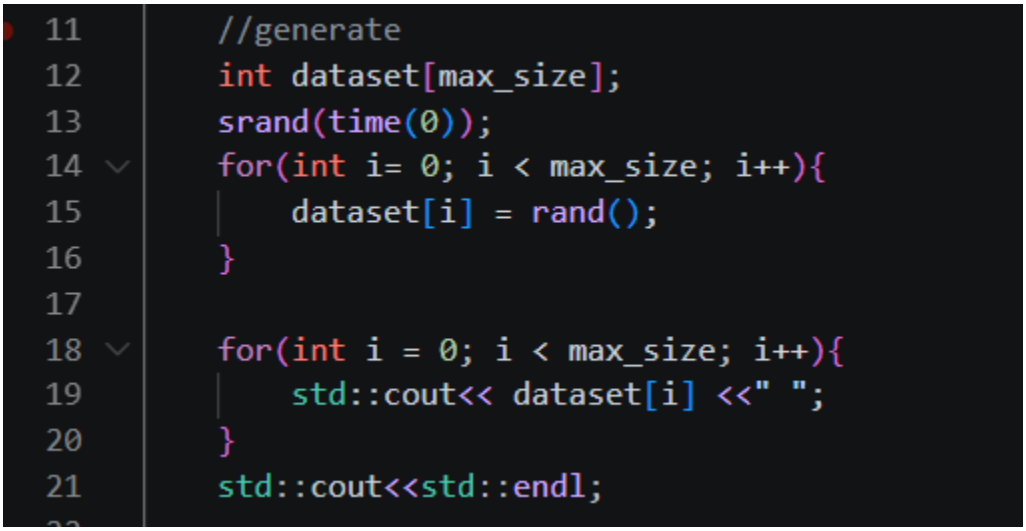
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	8/26/2026	**DATE SUBMITTED:**	8/26/2026
NAME:	Gabriel Immanuel R. Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

Screenshot	
Observation	Based on this code, we created a dataset array that stores the generated random numbers up to max_size of 50. Then we use srand(time(0)) to ensure that everytime we run the program it will generate random numbers that are different from the last run, and it uses a for loop to generate, store, and display all of the random numbers in the array.

```
11      //generate
12      int dataset[max_size];
13      srand(time(0));
14      for(int i= 0; i < max_size; i++){
15          dataset[i] = rand();
16      }
17
18      for(int i = 0; i < max_size; i++){
19          std::cout<< dataset[i] <<" ";
20      }
21      std::cout<<std::endl;
```

Table 6-1. Data Generated and Observations.

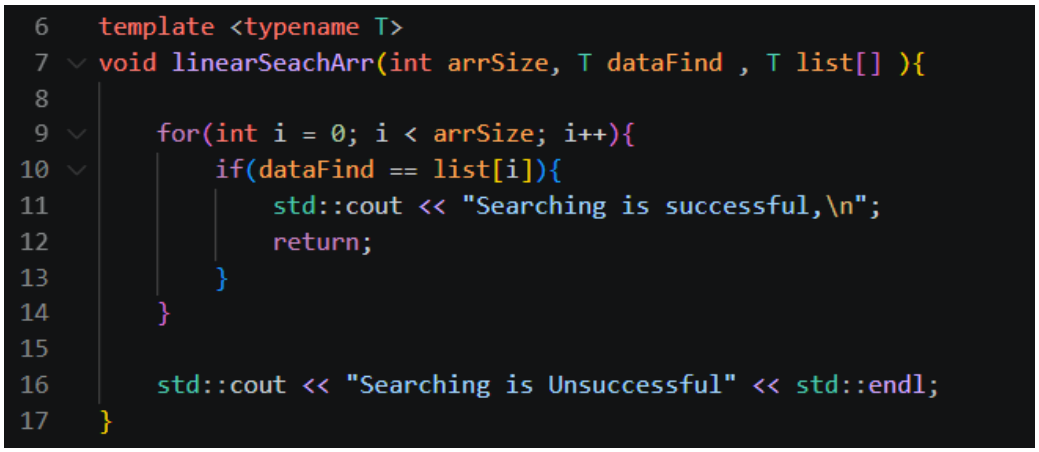
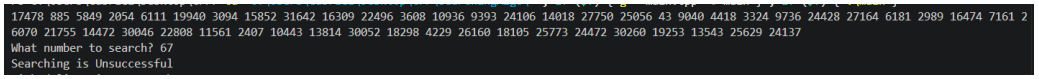
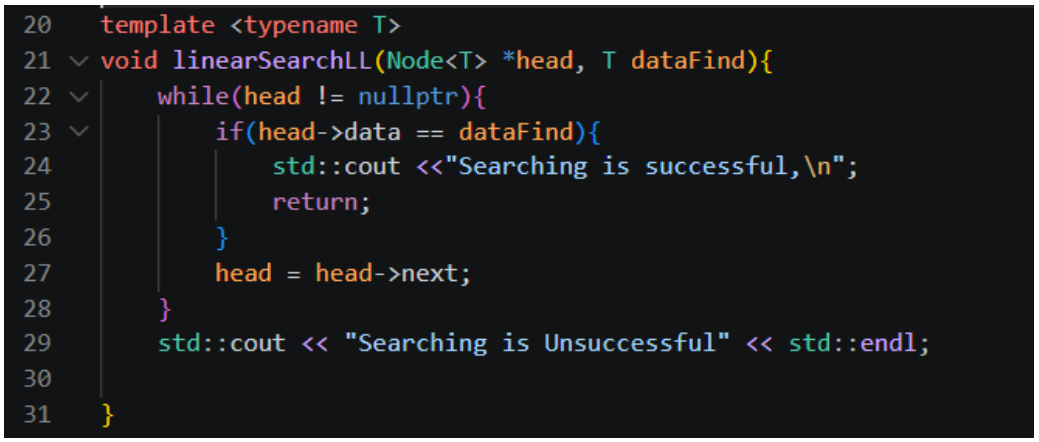
Screenshot	
Output	
Observation	Based on this code, we created a templated linearSearchArr function that searches for a specific value in an array. It works by using a for loop to check each element and compare it with the value we are trying to search for. If the value is found, it will display that the search is successful, if not, it will display that the search is unsuccessful. For example, if we search for the number 67, it will go through each index of the array until it finds the matching value.

Table 6-2a. Linear Search for Arrays

Screenshot	

```
29      //create linklist:
30      Node<char> * name1 = new_node('G');
31      Node<char> * name2 = new_node('A');
32      Node<char> * name3 = new_node('B');
33
34      name1->next = name2;
35      name2->next = name3;
36      name3->next = nullptr;
37
38
39      std::cout <<"Linked list Linear Search :";
40      linearSearchLL(name1, 'B');
41      std::cout <<"Linked list Linear Search :";
42      linearSearchLL(name1, 'C');
```

Output

```
Linked list Linear Search :Searching is successful,
Linked list Linear Search :Searching is Unsuccessful
```

Observation

Based on this code, we created a templated linearSearchLL function that searches for a specific value or character in a linked list. It works by using a while loop to check each node and compare its data with the value we are trying to search. For example I created 3 nodes that say GAB, it will go through each node of the linked list until it finds the matching value of B, and if there is none it will return unsuccessful.

Table 6-2b. Linear Search for Linked List

Screenshot

```
76  template <typename T>
77  void binarySearchArr(int arrSize, T dataFind, T list[]){
78
79      int low = 0, up = arrSize-1, middle;
80
81      while( low <= up){
82          middle = (int)((low+up)/2);
83
84          if (dataFind == list[middle]){
85              std::cout << "The search element is found \n";
86              return;
87          }
88          else if( dataFind < list[middle]){
89              up = middle-1;
90          }else{
91              low = middle+1;
92          }
93      }
94      std::cout << "The search element is not found \n";
95  }
```

```
44      //creating an array binary
45      int sortedArr[7] = {1,2,3,4,5,6,7};
46
47      std::cout << "Array Binary Search: ";
48      binarySearchArr(8,6, sortedArr);
49      std::cout << "Array Binary Search: ";
50      binarySearchArr(8,10, sortedArr);
```

Output

```
Array Binary Search: The search element is found
Array Binary Search: The search element is not found
```

Observation

Based on this code, we created a templated binarySearchArr function that searches for a specific value in a sorted array. It works by checking the low, up, and middle values to divide the array into two parts and to check the middle element we add the up and low and divide it by 2. For example, when searching for the 6, it divides the array and checks the middle value until it finds 6.

Table 6-3a. Binary Search for Arrays

Screenshot

```cpp
template <typename T>
Node<T>* getMiddle(Node<T> *start, Node<T> *last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){

        slow = slow->next;

        fast = fast->next->next;
    }

    return slow;
}
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //last node is set to NULL.
    Node<T>* last = nullptr;

    //Middle element is calculated using the two pointers
    while( start != last){

        Node<T>* middle = getMiddle(start, last);

        if(middle->data == dataFind){
            std::cout << "The data was found!\n";
            return;
        }

        if(middle->data > dataFind){
            last = middle;
        }
        else{
            start = middle->next;
        }
    }

    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

```
53      //add your main driver code here
54
55      Node<int> * value1 = new_node(10);
56      Node<int> * value2 = new_node(20);
57      Node<int> * value3 = new_node(30);
58
59      value1->next = value2;
60      value2->next = value3;
61      value3->next = nullptr;
62
63
64      std::cout << "Linked list Binary Search :";
65      binarySearchLL(value1, 30);
66
67      std::cout << "Linked list Binary Search :";
68      binarySearchLL(value1, 29);
```

Output

```
Linked list Binary Search :
The data was found!
Linked list Binary Search :
Searching is Unsuccessful
```

Observation

Based on this code, we created two functions. First is the templated binarySearchLL function that searches for a specific value in a linked list. The second is the getMiddle function, which is used by binarySearchLL to find the middle node in the linked list. For example, when we create three nodes with different values, it compares the middle value with the value we are searching for and splits the search by adjusting the start and last until it finds the specific value.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
//problem 1
template <typename T>
int sequentialSearchArr(int arrSize, T dataFind, T list[]){
    int comparison = 0;
    for(int i = 0; i < arrSize; i++){
        comparison++;
        if(dataFind == list[i]){
            std::cout <<"Searching is Successful";
            return comparison;
        }
    }
    std::cout <<"Searching is Unsuccessful";
    return comparison;
}

template <typename T>
int sequentialSearchLL(Node<T>* head, T dataFind){
    int comparison = 0;
    while(head != nullptr){
        comparison++;
        if(dataFind == head->data){
            std::cout <<"Searching is Successful";
            return comparison;
        }
        head = head->next;
    }
    std::cout <<"Searching is Unsuccessful";
    return comparison;
}
```

```cpp
//Problem 1
//array
int sequentialArr[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
std::cout << "Array Sequential Search: ";
int arrayComparisons = sequentialSearchArr(10, 18, sequentialArr);
std::cout << "\nTotal comparisons: " << arrayComparisons;

//linked list
Node<int> *value1 = new_node(15);
Node<int> *value2 = new_node(18);
Node<int> *value3 = new_node(2);
Node<int> *value4 = new_node(19);
Node<int> *value5 = new_node(18);
Node<int> *value6 = new_node(0);
Node<int> *value7 = new_node(8);
Node<int> *value8 = new_node(14);
Node<int> *value9 = new_node(19);
Node<int> *value10 = new_node(14);

value1->next = value2;
value2->next = value3;
value3->next = value4;
value4->next = value5;
value5->next = value6;
value6->next = value7;
value7->next = value8;
value8->next = value9;
value9->next = value10;
value10->next = nullptr;

std::cout << "\nLinked List Sequential Search: ";
int llComparisons = sequentialSearchLL(value1, 18);
std::cout << "\nTotal comparisons: " << llComparisons;
```

Output

```
Array Sequential Search: Searching is Successful
Total comparisons: 2
Linked List Sequential Search: Searching is Successful
Total comparisons: 2
```

Answers

Based on the question, both the array and the linked list have a total of two comparisons before the key 18 is found. When we created the array and linked list, the value 18 was placed at the second element of the array and the second node of the linked list. So after detecting that it found the value 18 it automatically returned and didn't check the inner of those list and array because the condition was to find the key 18 and there are many 18 there so it returns the first one it finds.

Code

```cpp
//problem 2
template <typename T>
int repetitionSearchArr(int arrSize, T dataFind, T list[]){
    int count = 0;
    for(int i = 0; i < arrSize; i++){
        if(dataFind == list[i]){
            count++;
        }
    }
    return count;
}

template <typename T>
int repetitionSearchLL(Node<T>* head, T dataFind){
    int count = 0;
    while(head != nullptr){
        if(dataFind == head->data){
            count++;
        }
        head = head->next;
    }
    return count;
}
```

```cpp
//problem 2
int arrCount = repetitionSearchArr(10, 18, sequentialArr);
std::cout << "\nThe repetition of 18 in the Array is: " << arrCount;
int llCount = repetitionSearchLL(value1, 18);
std::cout << "\nThe repetition of 18 in the linked list is: " << llCount;
```

Output

```
The repetition of 18 in the Array is: 2
The repetition of 18 in the linked list is: 2
```

Answers

Based on the question, both the array and the linked list have a total of two repetitions of the value 18. When we created the array and linked list, there were a total of two 18 in the linked list and the array. This time the function checks the whole array and the linked list and everytime that it finds the number 18 it increments the count by 1, so in total we have 2 repetition of the value 18.

PROBLEM 2

Diagram


Start
End
3
5
6
8
11
12
14
15
17
18
Low
11 is greater than 8 so we minus our up mid -1 and it will end up on the 8, and all the other value that is greater than 11 is canceled out
Middle
Now we calculate the middle by using the middle = (int)((low+up))/2 which gives us the middle of index 1
up
Start
End
3
5
6
8
11
12
14
15
17
18
Low
Middle
up
Now that our middle is less than 8 we do the same process again which cancels out the value less than 5 and move the middle up by 1


Start
End
3
5
6
8
11
12
14
15
17
18
Middle
up
Low
Same process here we also add the middle with 1 since 6 is still less than 8
Start
End
3
5
6
8
11
12
14
15
17
18
up
Middle
Low
And then we finally got the value 8

Code

```
61  //problem 3
62  template <typename T>
63  void binarySearchAlgo(int arrSize, T dataFind, T list[]){
64      int low = 0, up = arrSize -1, middle;
65
66      while(low <= up){
67          middle = (int)((low+up)/2);
68          if (dataFind == list[middle]){
69              std::cout <<"The element was found";
70              return;
71          }else if(dataFind < list[middle]){
72              up = middle - 1;
73          }else{
74              low = middle +1;
75          }
76      }
77      std::cout <<"The element was not found";
78  }
```

```
51      //problem 3
52      int binaryList[10] = {3,5,6,8,11,12,14,15,17,18};
53      std::cout << "\nArray: ";
54      binarySearchAlgo(10, 8, binaryList);
```

Output

```
The repetition of 18 in the linke
Array: The element was found
```

Answer

Based on the question, we created a templated binarySearchAlgo function that searches for the key 8 in a sorted array. This works by using the low, up ,and middle values to find the middle element and narrows it down to what we are trying to find. For the first iterations, all the values that are greater than 11 were crossed out and the middle moves to the number 5 and then it checks again now that it is less than it removes all the values less than 5 and moves to 6. Lastly it checks again if 6 is equal to 8 if not then it moves up again by 1 and then we get the 8 value.

Code

```
80  //problem 4
81  template <typename T>
82  void binarySearchAlgoRecursive(T list[], int low, int up, T dataFind){
83      if(low>up){
84          std::cout <<"The element was not found";
85          return;
86      }
87      int middle = (int)((low+up)/2);
88      if(dataFind == list[middle]){
89          std::cout << "The element was found";
90          return;
91      }else if( dataFind < list[middle]){
92          binarySearchAlgoRecursive(list, low, middle-1, dataFind);
93      }else{
94          binarySearchAlgoRecursive(list ,middle+1, up, dataFind);
95      }
96  }
97  #endif
```

```
56      //problem 4
57      std::cout << "\nArray recursive: ";
58      binarySearchAlgoRecursive(binaryList, 0, 9 ,12);
59      return 0;
```

Output

```
Array recursive: The element was found
```

Answer

Based on the question, we created a templated binarySearchAlgoRecursive function that performs a binary search using recursion. It works by checking the low, up, and middle values and calling the function again with a smaller search range depending if the key is greater or less than our middle value. For example, when searching for the value 12, it checks the middle value and calls the functions over and over again until it finds the value 12.

Conclusion :

I learned how to implement and understand how searching algorithms work by using arrays and linked lists. Through this activity, I somehow became familiar with the sequential search and binary search, like how they work and how you can count the comparisons that are made in order to find the specific value. I also learned how to modify the sequential search to count the repeated values on a given array and list. The most challenging part for me was the recursive part because it was difficult to understand how the function should work by calling itself and how to narrow it down properly using the up and lows. Overall this activity helped me improve my knowledge in understanding searching algorithms.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used on some part of the sentence types while ChatGPT helped me understand on how to draw the diagram that was used in the binary searching algorithms**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.